

PORTADA

Análisis y Diseño de Sistemas II

Semana 5 — Introducción al Diseño Arquitectural

Universidad de Antioquía · Ingeniería de Sistemas



ENCUADRE

De qué existe a cómo se organiza

La Unidad 2 cerró con el **modelo de análisis**: qué clases de dominio existen, cómo se relacionan y cómo el sistema reacciona a eventos externos (DSS). Esa vista describe el **problema**.

La Unidad 3 abre la pregunta de la solución: ¿cómo se organiza el software que va a resolver ese problema? A esa organización la llamamos *arquitectura de software*, y es el tema de las próximas dos semanas.

APERTURA

Un proceso Rails para todo



CASO · FICHA TÉCNICA

Twitter, 2006–2010

Sistema	Twitter — red social de microblogging
Arquitectura original	Aplicación monolítica en Ruby on Rails, un solo proceso para todo
Síntoma público	La "Fail Whale" — página de error mostrada durante caídas frecuentes, 2007–2010
Crecimiento	De unos pocos miles de tweets/día en 2007 a cientos de millones años después
Respuesta	Rearquitectura progresiva: del monolito a servicios independientes sobre la JVM (2010–2013)
Fuente	Charlas públicas de ingeniería de Twitter (ej. Raffi Krikorian, "Twitter's Architecture") y su blog de ingeniería

CASO · QUÉ HABÍA DENTRO

Todo en un mismo proceso

La primera versión de Twitter atendía en un **único proceso Rails** tareas que hoy consideraríamos completamente distintas: recibir un tweet nuevo, calcular a quién mostrárselo, generar la línea de tiempo de cada usuario, enviar notificaciones, servir la página web.

Ninguna de esas tareas podía escalar, desplegarse o fallar por separado — si una se saturaba, arrastraba a todas las demás. No era un error de código: era la **estructura** del sistema.

CASO · LA REARQUITECTURA

De un proceso a varios servicios

Entre 2010 y 2013, los equipos de ingeniería de Twitter separaron el monolito en **servicios independientes** sobre la JVM (Scala/Java), cada uno con una responsabilidad clara — por ejemplo, un servicio dedicado a datos de usuario y otro a la generación de líneas de tiempo.

El comportamiento visible para el usuario cambió poco. Lo que cambió fue la estructura interna: cómo se dividían las responsabilidades y cómo se comunicaban entre sí.

TRANSICIÓN

¿Qué fue exactamente lo que cambió?

Nadie reescribió "el código de Twitter" desde cero. Lo que cambió fue algo más abstracto: **la forma en que las piezas del sistema estaban organizadas y relacionadas entre sí.**

A ese "algo más abstracto" es a lo que esta unidad llama **arquitectura de software** — y hoy le ponemos definición formal.



CONCEPTO

Cuatro definiciones formales

Autor(es)	Definición, en síntesis
Perry & Wolf (1992)	Arquitectura = {Elementos, Formas, Restricciones} — de proceso, de datos, de conexión
Garlan & Shaw (1994)	La organización del sistema como conjunto de componentes , sus conectores y sus patrones de interacción
Bass, Clements & Kazman (SEI)	El conjunto de estructuras necesarias para razonar sobre el sistema: elementos, relaciones y propiedades de ambos
ISO/IEC/IEEE 42010	Los conceptos fundamentales de un sistema, encarnados en sus elementos, relaciones y principios de diseño y evolución

Las cuatro coinciden en algo: elementos + relaciones + una razón para existir más allá del dibujo.

CONTRAEJEMPLO

Mito: "la arquitectura es el diagrama"

Es común confundir la arquitectura con su representación gráfica: el diagrama de cajas y flechas que se dibuja en una herramienta.

El diagrama es una vista de la arquitectura, no la arquitectura misma. Twitter tuvo arquitectura —buena o mala— incluso en 2007, cuando nadie había dibujado un diagrama formal: la arquitectura es la estructura real que existe en el sistema en ejecución, se haya documentado o no.

CONCEPTO

¿Qué hace que una decisión sea "arquitectónica"?

No toda decisión de programación es una decisión de arquitectura. Una decisión es **arquitectónica** cuando cumple varias de estas condiciones:

- Es **costosa o difícil de revertir** una vez tomada.
- Afecta a **múltiples componentes** del sistema, no a uno aislado.
- Compromete **atributos de calidad** del sistema completo: rendimiento, escalabilidad, seguridad, mantenibilidad.
- Restringe las decisiones que se pueden tomar después.



CONCEPTO

Arquitectura vs. diseño de detalle, lado a lado

	Arquitectura	Diseño de detalle
Alcance	El sistema completo	Un componente o módulo
Costo de cambiar	Alto — a veces rehacer el proyecto	Bajo a moderado — se ajusta sin rehacer todo
Cuándo se decide	Muy temprano, con menos información	A lo largo de todo el desarrollo
Ejemplo	Monolito vs. servicios independientes	El algoritmo de ordenamiento de una lista

Lo que separa arquitectura de diseño no es cuánto código involucra, sino qué tan costoso es dar marcha atrás.

CONCEPTO

Atributos de calidad: lo que la arquitectura compromete

Un **atributo de calidad** es una propiedad medible del sistema completo, más allá de si "hace lo que debe hacer" (eso lo cubre el modelo de análisis). La arquitectura es la que determina cuánto de cada atributo es alcanzable.

Atributo	Pregunta que responde
Rendimiento	¿Qué tan rápido responde bajo carga normal?
Escalabilidad	¿Aguanta crecer en usuarios o datos sin rediseñarse?
Disponibilidad	¿Sigue funcionando si una parte falla?
Seguridad	¿Resiste accesos e intenciones no autorizadas?
Mantenibilidad	¿Qué tan barato es modificarlo después?



TENSIÓN CONCEPTUAL

Los atributos de calidad compiten entre sí

Optimizar un atributo casi siempre cuesta algo de otro. Ejemplos frecuentes:

- **Seguridad vs. rendimiento:** más capas de validación y cifrado, más tiempo de respuesta.
- **Disponibilidad vs. consistencia:** replicar datos en varios servidores para no caerse implica que, por un instante, no todos vean lo mismo.
- **Mantenibilidad vs. rendimiento:** el código más fácil de leer no siempre es el más rápido de ejecutar.

No existe una arquitectura que maximice todos los atributos a la vez — toda decisión arquitectónica es un compromiso explícito.

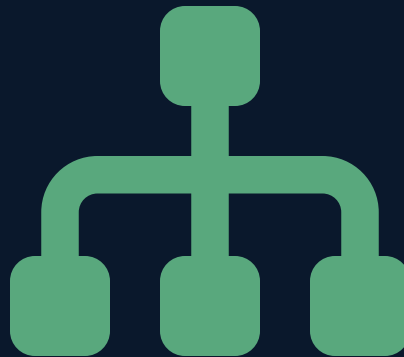


CONCEPTO

La Ley de Conway (1967)

Las organizaciones que diseñan sistemas están limitadas a producir diseños que son copias de las estructuras de comunicación de esas organizaciones.

En otras palabras: **si tres equipos se comunican poco entre sí, es muy probable que construyan tres módulos poco integrados** — la estructura social se filtra, casi siempre sin querer, en la estructura técnica. En Twitter, cada servicio nuevo terminó teniendo un equipo propio responsable.



INTERACCIÓN

¿Arquitectura o de diseño?

Clasifiquen cada decisión y justifiquen con el criterio de "costo de revertir":

1. Elegir si el sistema del proyecto va a ser un monolito o estar dividido en varios servicios.
2. Elegir qué estructura de datos usar para ordenar una lista dentro de un solo método.
3. Decidir que la aplicación va a exponer sus funciones como una API REST, en vez de embeber la lógica en el frontend.
4. Cambiar el algoritmo de validación de un formulario de registro.

5 minutos · respondan en el chat o en voz alta

CONCEPTO EN DISPUTA

¿Toda la arquitectura se diseña por adelantado?

Postura clásica (Big Design Up Front): la arquitectura se define completa antes de escribir la primera línea de código de negocio.

Postura ágil/evolutiva: Twitter no diseñó en 2006 la arquitectura de servicios que tuvo en 2013 — la arquitectura **emergió y se transformó** a medida que el sistema creció, con decisiones tomadas cuando hubo evidencia real de que el diseño original no aguantaba.

No hay consenso único en la industria sobre cuánto diseñar por adelantado y cuánto dejar evolucionar — depende del riesgo, el dominio y qué tan cara es la reversión de cada decisión.

CONCEPTO · APLICADO AL PROYECTO

Por qué importa esta semana, exactamente

La arquitectura no es un ejercicio académico separado del código: es lo que determina si el proyecto de Fábrica Escuela va a **aguantar cambios, crecer y mantenerse** sin reescribirse cada sprint.

- Una mala decisión arquitectónica tomada en el Sprint 1 puede ser muy costosa de corregir en el Sprint 2 o 3.
- El viernes vemos los **estilos de arquitectura** disponibles (monolítica, cliente/servidor, SOA, microservicios, entre otros) para que cada equipo elija con criterio, no por intuición.
- La semana 6 formaliza esas decisiones con **diagramas UML de despliegue, paquetes y componentes** — el entregable arquitectónico del Sprint 1.

SÍNTESIS

Ideas clave de hoy

1. La arquitectura es la **estructura** del sistema —elementos, relaciones y principios de diseño y evolución— no un diagrama.
2. Una decisión es **arquitectónica** si es costosa de revertir, afecta a varios componentes o compromete atributos de calidad.
3. Los **atributos de calidad** (rendimiento, escalabilidad, disponibilidad, seguridad, mantenibilidad) compiten entre sí: toda arquitectura es un compromiso explícito.
4. La **Ley de Conway** conecta la estructura del sistema con la estructura de quienes lo construyen.
5. No hay consenso sobre diseñar toda la arquitectura por adelantado — Twitter es evidencia de arquitectura que evolucionó bajo presión real.



CIERRE

Para el viernes

Traigan identificado, en su propio proyecto de Fábrica Escuela, un atributo de calidad que consideren **crítico** (por ejemplo: disponibilidad si el sistema es transaccional, o escalabilidad si esperan muchos usuarios concurrentes).

El viernes recorreremos los estilos de arquitectura disponibles y cada equipo empieza a decidir cuál se ajusta mejor a su proyecto — insumo directo para el Planning Sprint 1 del Coworking de esta semana.

